Отчет по лабораторной работе 6

Написана программа point_to_point.c с тремя режимами. basic — процесс 0 отправляет процессу 1 строку «Hello from rank 0» (MPI_Send), процесс 1 принимает её (MPI_Recv), печатает и отвечает «Message received». all — процесс 0 в цикле по рангам 1…p−1 рассылает каждому своё сообщение «Message for rank i», затем p−1 раз принимает ответ от MPI_ANY_SOURCE и по status.MPI_SOURCE и тексту проверяет, что каждый ранг ответил ровно один раз и получил именно своё сообщение. bench — ping-pong между 0 и 1 с массивом символов разного размера. Коллективные операции не используются; при p < 2 программа печатает ошибку и завершается через MPI_Finalize.

Схема и теги: каждый вызов явно задаёт буфер, число элементов (длина строки + 1 или размер массива), тип MPI_CHAR, ранг получателя/отправителя и тег: TAG_MESSAGE = 1 — сообщение от 0 к рабочему процессу, TAG_REPLY = 2 — ответ обратно к 0. Разные теги разделяют два направления: приём ответа не перепутается с сообщением того же отправителя.

Порядок обмена: P0 —Send(TAG_MESSAGE)→ P1: Recv, печать; P1 —Send(TAG_REPLY)→ P0: Recv, печать. Порядок «Send, затем Recv» у 0 и «Recv, затем Send» у 1 согласован, поэтому взаимной блокировки нет.

Система: Intel Core i5-9400F (6 ядер), WSL2 Ubuntu 26.04, Open MPI 5.0.10, gcc 15.2; все процессы на одном узле, обмен идёт через общую память.

Команды: mpicc -O2 -Wall -Wextra point_to_point.c -o point_to_point; mpiexec -n 2 ./point_to_point basic; mpiexec -n 4 ./point_to_point all; mpiexec --oversubscribe -n 8 ./point_to_point all; mpiexec -n 2 ./point_to_point bench (8 процессов > 6 ядер, поэтому нужен --oversubscribe).

3.1. Базовый обмен

Запуск	p	Сообщение принято?	Ответ получен?	Завершилась?
1	2	да	да	да (код 0)
2	2	да	да	да (код 0)
3	2	да	да	да (код 0)

3.2. Обмен 0 ↔ все процессы

p	Отправлено	Получено ответов	Все верны?
2	1	1	да
4	3	3	да
6	5	5	да
8*	7	7	да

* 8 процессов на 6 ядрах — переподписка; обмен корректен, просто процессы делят ядра. Зависаний не было ни в одном запуске (каждый запуск под timeout 30 с).

3.3. Ping-pong 0 ↔ 1 (100 пробных обменов, затем медиана 5 серий; размер проверен через MPI_Get_count)

Размер, байт	Повторений	Время туда-обратно, с	То же, мкс	Скорость 2m/t, ГБ/с	Размер принят верно?
1	100000	$3.56 \cdot 10^{-7}$	0.36	0.01	да
1024	100000	$8.41 \cdot 10^{-7}$	0.84	2.44	да
65536	10000	$1.58 \cdot 10^{-5}$	15.80	8.29	да
1048576	1000	$1.81 \cdot 10^{-4}$	181.50	11.55	да

Вывод:

1) В MPI процесс адресуется рангом в коммуникаторе (здесь MPI_COMM_WORLD, ранги 0…p−1): в MPI_Send указывается ранг получателя dest, в MPI_Recv — ранг отправителя source или MPI_ANY_SOURCE, а фактический отправитель читается из status.MPI_SOURCE. Ранг — номер процесса, а не ядра или узла. 2) Тег — целая метка сообщения: приём срабатывает, только если совпадают коммуникатор, отправитель и тег. Поэтому сообщения разного смысла от одного процесса (запрос и ответ) не путаются, а приём по тегу TAG_REPLY с MPI_ANY_SOURCE собирает только ответы. 3) Базовый обмен трижды прошёл без зависаний. Строки разных процессов в выводе могут идти в любом порядке, потому что mpiexec пересылает их вывод независимо, а порядок самого обмена задают блокирующие Send/Recv. 4) Рассылка в цикле работает при p = 2…8: каждый рабочий процесс получил своё сообщение, процесс 0 получил p−1 ответов от разных рангов. 5) Время обмена описывается моделью $t \approx \alpha + m/\beta$: до 1 КБ доминирует задержка (0.36–0.84 мкс туда-обратно, ≈ 0.2 мкс в одну сторону через общую память), а с 64 КБ — копирование данных: скорость растёт до ≈ 11.6 ГБ/с при 1 МБ. Мелкие сообщения выгодно объединять в крупные.